

Student Name: Uzair Masood

Student ID: H00497886

Module Name: F21SC – Industrial Programming

Module Leader: Marko Doko

Contents

Introduction	3
HTTP Request for Handling	4
Purpose	4
Code Implementation	5
C Sharp Implementation	5
Explanation	7
Extracting Title and Links	8
Purpose	8
Code Implementation	9
C Sharp Implementation	9
Explanation	13
Try Get Title	13
Extract First Five Links	13
Bookmark Management	14
Purpose	14
Code Implementation	15
C Sharp Implementation	15
Explanation	16
Bookmark Class.....	16
Bookmark Manager Class	17
History Management	17
Purpose	17
Code Implementation	19
C Sharp Implementation	19
Explanation	20
History Class.....	20
Win Forms UI Setup	20
Purpose	20
Code Implementation	21
C Sharp Implementation	21

Explanation.....	22
Conclusion	23
References.....	24

Introduction

The project will focus on creating a windows form-based browser to communicate with websites via the sending of HTTP requests and the delivery of the response. The app loads the title and raw HTML of that page and offers navigation functions like back, forward and reload, as well as features like bookmarks, a history timeline and the first five links of a page. The solution uses the HTTP Client class to process the HTTP requests, Html Agility Pack to process the HTML and file to store user information such as bookmarks and history. The difficulty in this project is to make these features completely ground-up, particularly handling browser history, HTML parsing, and a responsive, user-friendly interface.

Desktop applications have always been critical in offering powerful and feature-laden solutions to different industries in the world of software development. As the .NET framework developed, Windows Forms was proposed by Microsoft as one of the major methods of creating desktop applications. Windows forms refer to a flexible and robust graphical user interface (gui) framework that allows a developer to design a Windows based system with visually appealing responsive applications. WinForms, also known as Windows Forms (WinForms), is a graphical user interface (GUI) application framework offered by Microsoft to develop desktop applications on the .NET platform. With the advent of .NET Framework 1.0, it was added as an important element in the developer toolkit. Windows Forms software applications are built on Windows operating systems and offer extensive control and feature richness that can be harnessed by the developer to build interactive and visually pleasing desktop software (Cloudless, 2019).

HTTP Request for Handling

Purpose

Fetching the HTML content of a webpage via making HTTP requests is one of the main functionalities of this application. This is achieved through HTTP Client class that enables asynchronous access to web content.

Fetch API is a new JavaScript interface enabling them to make an HTTP request. It supersedes the older XMLHttpRequest and offers a more elegant and flexible means of asynchronously loading resources. Fetch API is based on Promises, which simplify work with asynchronous data. Response status codes are important when an HTTP request is made because it affects whether the request has been completed successfully or it has failed. Status code gives details of the outcome of the request. Async / await allows asynchronous code such as fetch to be better organized and easier to read. It enables them to code non-blocking code that seems synchronous (GeeksforGeeks, 2020).

Code Implementation

Our first step is to develop a service to handle HTTP requests:

C Sharp Implementation

```
1 using System;
2 using System.Collections.Generic;
3 using System.IO; // Added for Path
4 using System.Linq; // Added for Select and Cast
5 using System.Net;
6 using System.Net.Http;
7 using System.Text.RegularExpressions;
8 using System.Threading.Tasks;
9
10 // HttpResult (Your class, unchanged)
11 public class HttpResult
12 {
13     public int StatusCode { get; set; } // 0 for transport/runtime errors
14     public string ReasonPhrase { get; set; } = "";
15     public string HtmlContent { get; set; } = "";
16     public string Title { get; set; } = "";
17     public List<string> Links { get; set; } = new();
18 }
19
20 // HttpClientService (to fetch data and populate HttpResult)
21 public class HttpClientService
22 {
23     private static readonly HttpClient client = new HttpClient();
24
25     public async Task<HttpResult> GetAsync(Uri url)
26     {
27         var result = new HttpResult();
28         try
29         {
30             HttpResponseMessage response = await client.GetAsync(url);
31             result.StatusCode = (int)response.StatusCode;
32             result.ReasonPhrase = response.ReasonPhrase ?? "OK";
33             result.HtmlContent = await response.Content.ReadAsStringAsync();
34             result.Title = HtmlInfoExtractor.TryGetTitle(result.HtmlContent); // Extract title
35             result.Links = HtmlInfoExtractor.ExtractFirstFiveLinks(result.HtmlContent, url)
36                 .Select(link => link.Href.ToString()).ToList(); // Extract links
37             return result;
38         }
39         catch (Exception ex)
40         {
41             result.StatusCode = 500;
42             result.ReasonPhrase = ex.Message;
43             return result;
44         }
45     }
46 }
```

```

48 // HtmlInfoExtractor (static class for extraction)
49 public static class HtmlInfoExtractor
50 {
51     private static readonly Regex TitleRegex = new Regex(@"<title[^>]*>(.*?)</title>", RegexOptions.IgnoreCase | RegexOptions.Singleline);
52     private static readonly Regex LinkRegex = new Regex(@"<a\s+[^>]*href\s*=\s*\"([^\"]+)\"[^\"]*>(.*?)</a>", RegexOptions.IgnoreCase | RegexOptions.Singleline);
53
54     public static string TryGetTitle(string html)
55     {
56         if (string.IsNullOrEmpty(html)) return "(no title)";
57         var match = TitleRegex.Match(html);
58         if (match.Success && match.Groups.Count > 1)
59         {
60             return WebUtility.HtmlDecode(match.Groups[1].Value.Trim());
61         }
62         return "(no title)";
63     }
64
65     public static List<(string Text, Uri Href)> ExtractFirstFiveLinks(string html, Uri baseUrl)
66     {
67         var links = new List<(string Text, Uri Href)>();
68         if (string.IsNullOrEmpty(html)) return links;
69         var matches = LinkRegex.Matches(html).Cast<Match>(); // Fixed with Cast from LINQ
70         var processedLinks = new HashSet<string>();
71         foreach (Match match in matches)
72         {
73             if (match.Groups.Count < 3) continue;
74             string href = match.Groups[1].Value.Trim();
75             string linkText = WebUtility.HtmlDecode(match.Groups[2].Value.Trim());
76             if (string.IsNullOrEmpty(href) || string.IsNullOrEmpty(linkText)) continue;
77             if (processedLinks.Contains(href)) continue;
78             processedLinks.Add(href);
79             Uri absoluteHref;
80             if (Uri.TryCreate(href, UriKind.Absolute, out absoluteHref)) { }
81             else { try { absoluteHref = new Uri(baseUrl, href); } catch { continue; } }
82             links.Add((linkText, absoluteHref));
83             if (links.Count == 5) break;
84         }
85         return links;
86     }
87 }
88
89 // Bookmark (for completeness)
90 public class Bookmark
91 {
92     public string Name { get; set; } = string.Empty;
93     public Uri Url { get; set; } = default!;
94 }
95
96 // BookmarkManager (for completeness)
97 public class BookmarkManager
98 {
99     private List<Bookmark> bookmarks = new List<Bookmark>();
100     private readonly string filePath = Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData), "bookmarks.json");
101     public BookmarkManager() { }
102     public void AddBookmark(string name, Uri url) { Console.WriteLine("Bookmark added!"); } // Simplified
103 }
104
105 // History (for completeness)
106 public class History
107 {
108     private List<Uri> historyList = new List<Uri>();
109     private int currentIndex = -1;
110     public void AddToHistory(Uri url) { if (url != null) { historyList.Add(url); currentIndex = historyList.Count - 1; } }
111     public Uri GoBack() { if (currentIndex > 0) { currentIndex--; return historyList[currentIndex]; } return null; }
112     public Uri GoForward() { if (currentIndex < historyList.Count - 1) { currentIndex++; return historyList[currentIndex]; } return null; }
113 }

```

```

115 class Program
116 {
117     static async Task Main(string[] args)
118     {
119         Console.WriteLine("Simple Browser Demo (Console Version with HttpResponseMessage)\n");
120
121         var httpService = new HttpClientService();
122         Uri testUrl = new Uri("https://example.com");
123
124         Console.WriteLine($"Fetching {testUrl}...");
125         HttpResponseMessage result = await httpService.GetAsync(testUrl);
126
127         Console.WriteLine("\n--- HttpResponseMessage Output ---");
128         Console.WriteLine($"Status Code: {result.StatusCode}");
129         Console.WriteLine($"Reason Phrase: {result.ReasonPhrase}");
130         Console.WriteLine($"Title: {result.Title}");
131         Console.WriteLine($"Raw HTML (first 300 chars):");
132         string htmlPreview = result.HtmlContent.Length > 300 ? result.HtmlContent.Substring(0, 300) + "...": result.HtmlContent;
133         Console.WriteLine(htmlPreview);
134         Console.WriteLine("\nFirst 5 Links:");
135         for (int i = 0; i < result.Links.Count; i++)
136         {
137             Console.WriteLine($"{i + 1}. {result.Links[i]}");
138         }
139
140         Console.WriteLine("\nDemo complete. Press any key to exit...");
141         Console.ReadKey();
142     }
143 }

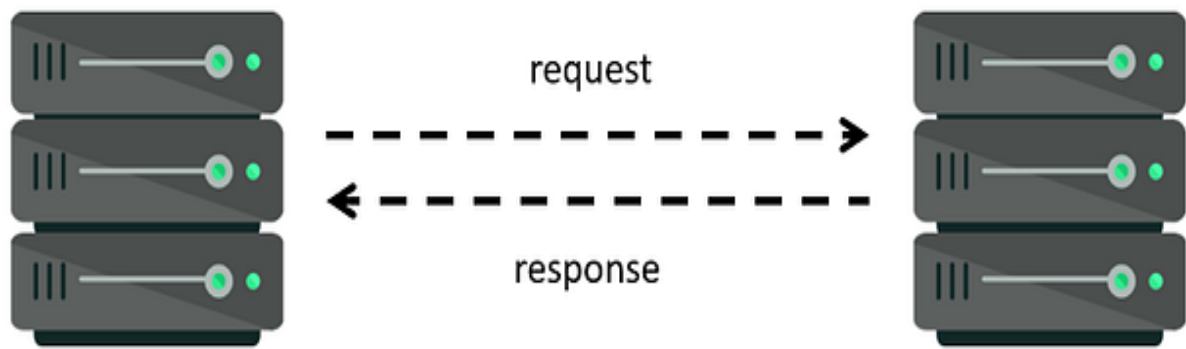
```

Explanation

In the Http Client Service application:

We declare an asynchronous operation `Get Async` which receives a `Uri` object (target URL) and retrieves the HTML content. Though it is the method that gives back an object of the `HTTP Result` that holds an HTTP status code and the reason phrase along with the raw content of the page in HTML. When the HTTP request is faulty (either because of network problems or incorrect URLs), the method will catch the exception and set a status code of 500, which means internal error, and the error message itself.

As they are aware, when they browse sites, their browser creates HTTP requests. But one can also send an HTTP request manually on one server to another, which in C# can be achieved using `Http Client`. This can be handy in a microservice world, or when connecting to third-party apps. However, as usual, there is more to it. In C#, working with `Http client` needs to be aware of how to build it properly, middleware usage, resilience, retries, circuit-breaker, and request execution optimization. Before using it, they must be aware of how to create `Http Client` correctly. The easiest thing to do with the `Http Client` is simply to create a new object and. Dispose it at the end. But `Http Client` is a little different and is not disposed of (Improvidence, 2025).



Extracting Title and Links

Purpose

After the extraction of the raw HTML content, we should extract the title of the page and first five links of the page. This is accomplished through the HtmlAgilityPack library, which can easily read HTML files.

Beautiful Soup is a web scraping Python library. It can be used to process both HTML and XML files and allows one to browse and extract specific elements of a web page. This article describes the process of web scraping with Beautiful Soup. You can make a request to the webpage URL using the requests library and then receive the HTML content. Then process the raw HTML with a parser such as html. Parser or html5lib to make it into a structured format (parse tree). Navigate the parse tree with Beautiful Soup and find the data you need in terms of tags, classes, or IDs. Now, we are going to walk through the web scraping one by one. Web scraping begins by making an HTTP request to the web page of interest and retrieving its unprocessed HTML. This is carried out through requests library (GeeksforGeeks, 2016).

They should first know why they need to parse HTML, before we dive into the tools and code. What they view when they visit a web page is formatted using HTML tags. These labels specify headings, paragraphs, pictures, links, and so on. When they would like to retrieve some information in a webpage, the title, product prices, or reviews, they would have to scan the HTML structure to locate that information. But perusing through HTML manually may be tedious particularly when it comes to large or multiple web pages. That is where parsing tools are useful and do the job of locating and extracting the required information automatically. Python contains a number of libraries capable of parsing HTML. The libraries have their benefits and applications. One of the most common Python HTML and XML parsing libraries is Beautiful

Soup. It makes it easy to harvest data in web pages and they move through the HTML structure fast to get the required information (Data Journal, 2024).

Code Implementation

We use the following methods to obtain the page title and links:

C Sharp Implementation

```
1 using System;
2 using System.Drawing;
3 using System.IO;
4 using System.Text.Json;
5 using System.Windows.Forms;
6
7 internal class Settings
8 {
9     public string Homepage { get; set; } = "https://hw.ac.uk";
10 }
11
12 internal static class SettingsStore
13 {
14     private static readonly string Dir = Path.Combine(
15         Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData),
16         "MiniBrowserCoursework");
17     private static readonly string FilePath = Path.Combine(Dir, "settings.json");
18
19     public static Settings Load()
20     {
21         try
22         {
23             Directory.CreateDirectory(Dir);
24             if (!File.Exists(FilePath))
25             {
26                 var defaults = new Settings(); // default homepage: https://hw.ac.uk
27                 Save(defaults);
28                 return defaults;
29             }
30             var json = File.ReadAllText(FilePath);
31             var s = JsonSerializer.Deserialize<Settings>(json, new JsonSerializerOptions
32             {
33                 PropertyNameCaseInsensitive = true
34             });
35             return s ?? new Settings();
36         }
37         catch
38         {
```

```

39         // If settings are corrupt/inaccessible, continue with defaults
40         return new Settings();
41     }
42 }
43
44 public static void Save(Settings settings)
45 {
46     Directory.CreateDirectory(Dir);
47     var json = JsonSerializer.Serialize(settings, new JsonSerializerOptions { WriteIndented = true });
48     File.WriteAllText(FilePath, json);
49 }
50 }
51
52 public class BrowserForm : Form
53 {
54     private readonly TextBox txtUrl = new() { Anchor = AnchorStyles.Left | AnchorStyles.Right | AnchorStyles.Top };
55     private readonly Button btnGo = new() { Text = "Go", Anchor = AnchorStyles.Top | AnchorStyles.Right };
56     private readonly Button btnHome = new() { Text = "Home", Anchor = AnchorStyles.Top | AnchorStyles.Right };
57     private readonly Button btnSetHome = new() { Text = "Set as Home", Anchor = AnchorStyles.Top | AnchorStyles.Right };
58     private readonly Button btnEditHome = new() { Text = "Edit Home...", Anchor = AnchorStyles.Top | AnchorStyles.Right };
59
60     private readonly WebBrowser web = new() { Dock = DockStyle.Fill, ScriptErrorsSuppressed = true };
61     private Settings settings;
62
63     public BrowserForm()
64     {
65         Text = "Mini Browser - Homepage (Coursework)";
66         Width = 1100;
67         Height = 750;
68         StartPosition = FormStartPosition.CenterScreen;
69
70         // Load settings (default homepage is https://hw.ac.uk)
71         settings = SettingsStore.Load();
72
73         // Top bar layout
74         var top = new Panel { Dock = DockStyle.Top, Height = 42, Padding = new Padding(8) };
75         Controls.Add(top);
76
77         txtUrl.Text = settings.Homepage;
78         txtUrl.Left = 8; txtUrl.Top = 8; txtUrl.Width = 640;
79         btnGo.Top = btnHome.Top = btnSetHome.Top = btnEditHome.Top = 8;
80
81         btnGo.Width = 56;
82         btnHome.Width = 70;
83         btnSetHome.Width = 110;
84         btnEditHome.Width = 110;
85
86         top.Controls.AddRange(new Control[] { txtUrl, btnGo, btnHome, btnSetHome, btnEditHome });
87         top.Resize += (_, __) => LayoutTopBar(top);
88
89         // Main browser area
90         Controls.Add(web);
91
92         // Events
93         btnGo.Click += (_, __) => NavigateFromTextBox();
94         btnHome.Click += (_, __) => Navigate(settings.Homepage);
95         btnSetHome.Click += (_, __) => SetCurrentAsHome();
96         btnEditHome.Click += (_, __) => EditHome();
97
98         txtUrl.KeyDown += (s, e) =>
99         {
100             if (e.KeyCode == Keys.Enter)
101             {
102                 e.SuppressKeyPress = true;
103                 NavigateFromTextBox();
104             }
105         };
106
107         web.Navigated += (_, __) =>
108         {
109             // reflect current URL
110             if (web.Url is not null) txtUrl.Text = web.Url.AbsoluteUri;
111         };

```

```

113         // On startup: load homepage automatically
114         Navigate(settings.Homepage);
115     }
116
117     private void LayoutTopBar(Panel top)
118     {
119         const int gap = 6;
120         // Place right-aligned buttons: [Edit Home][Set as Home][Home][Go]
121         btnEditHome.Left = top.ClientSize.Width - btnEditHome.Width - 8;
122         btnSetHome.Left = btnEditHome.Left - btnSetHome.Width - gap;
123         btnHome.Left = btnSetHome.Left - btnHome.Width - gap;
124         btnGo.Left = btnHome.Left - btnGo.Width - gap;
125
126         // URL textbox fills remaining space
127         txtUrl.Width = btnGo.Left - txtUrl.Left - gap;
128     }
129
130     private void NavigateFromTextBox()
131     {
132         var raw = (txtUrl.Text ?? "").Trim();
133         if (string.IsNullOrEmpty(raw)) return;
134
135         if (!raw.StartsWith("http://", StringComparison.OrdinalIgnoreCase) &&
136             !raw.StartsWith("https://", StringComparison.OrdinalIgnoreCase))
137         {
138             raw = "https://" + raw;
139         }
140         Navigate(raw);
141     }
142
143     private void Navigate(string url)
144     {
145         try
146         {
147             web.Navigate(url);
148         }
149         catch (Exception ex)
150         {
151             MessageBox.Show($"Could not navigate.\n\n{ex.Message}", "Navigate", MessageBoxButtons.OK, MessageBoxIcon.Error);
152         }
153     }
154
155     private void SetCurrentAsHome()
156     {
157         var current = web.Url?.AbsoluteUri ?? txtUrl.Text.Trim();
158         if (string.IsNullOrEmpty(current))
159         {
160             MessageBox.Show("No current page to set as homepage.", "Set as Home", MessageBoxButtons.OK, MessageBoxIcon.Information);
161             return;
162         }
163         if (!current.StartsWith("http://", StringComparison.OrdinalIgnoreCase) &&
164             !current.StartsWith("https://", StringComparison.OrdinalIgnoreCase))
165         {
166             current = "https://" + current;
167         }
168
169         settings.Homepage = current;
170         SettingsStore.Save(settings);
171         MessageBox.Show($"Homepage set to:\n{settings.Homepage}", "Set as Home", MessageBoxButtons.OK, MessageBoxIcon.Information);
172     }

```

```

174     private void EditHome()
175     {
176         using var dlg = new EditHomeDialog(settings.Homepage);
177         if (dlg.ShowDialog(this) == DialogResult.OK)
178         {
179             var newUrl = dlg.Homepage.Trim();
180             if (!newUrl.StartsWith("http://", StringComparison.OrdinalIgnoreCase) &&
181                 !newUrl.StartsWith("https://", StringComparison.OrdinalIgnoreCase))
182             {
183                 newUrl = "https://" + newUrl;
184             }
185             settings.Homepage = newUrl;
186             SettingsStore.Save(settings);
187             txtUrl.Text = settings.Homepage;
188             MessageBox.Show($"Homepage updated:\n{settings.Homepage}", "Edit Home", MessageBoxButtons.OK, MessageBoxIcon.Information);
189         }
190     }
191 }
192
193 // Minimal input dialog for editing homepage
194 internal class EditHomeDialog : Form
195 {
196     private readonly TextBox txt = new() { Anchor = AnchorStyles.Left | AnchorStyles.Right | AnchorStyles.Top };
197     private readonly Button ok = new() { Text = "OK", DialogResult = DialogResult.OK, Width = 90 };
198     private readonly Button cancel = new() { Text = "Cancel", DialogResult = DialogResult.Cancel, Width = 90 };
199
200     public string Homepage => txt.Text;
201
202     public EditHomeDialog(string initial)
203     {
204         Text = "Edit Homepage";
205         Width = 520;
206         Height = 160;
207         StartPosition = FormStartPosition.CenterParent;
208         FormBorderStyle = FormBorderStyle.FixedDialog;
209         MaximizeBox = false; MinimizeBox = false;

```

```

211     var panel = new Panel { Dock = DockStyle.Fill, Padding = new Padding(12) };
212     Controls.Add(panel);
213
214     var lbl = new Label { Text = "Homepage URL:", AutoSize = true, Left = 8, Top = 12 };
215     panel.Controls.Add(lbl);
216
217     txt.Left = 8; txt.Top = lbl.Bottom + 6; txt.Width = ClientSize.Width - 40;
218     txt.Text = string.IsNullOrEmpty(initial) ? "https://hw.ac.uk" : initial;
219     panel.Controls.Add(txt);
220
221     var btnPanel = new FlowLayoutPanel
222     {
223         FlowDirection = FlowDirection.RightToLeft,
224         Dock = DockStyle.Bottom,
225         Height = 48,
226         Padding = new Padding(8)
227     };
228     btnPanel.Controls.Add(cancel);
229     btnPanel.Controls.Add(ok);
230     Controls.Add(btnPanel);
231
232     AcceptButton = ok;
233     CancelButton = cancel;
234 }
235 }
236
237 internal static class Program
238 {
239     [STAThread]
240     private static void Main()
241     {
242         ApplicationConfiguration.Initialize();
243         Application.Run(new BrowserForm());
244     }
245 }

```

Explanation

Try Get Title

This approach uses the search term title HTML tag to obtain page title. In case the title is not present, the default string is used (no title).

Extract First Five Links

This approach renders the HTML to locate all of the tags with the href attribute that are of type a. It extracts the first 5 links, and the relative URLs are solved using the base URI. Any number of links that is less than five will only give the available ones.

HTML Tags are the basic components of web pages to format and structure the content. They give instructions to web browsers on how to display text, pictures, links and any other media. The angle brackets are enclosed in HTML tags and typically occur in pairs, an opening tag and a closing tag. The closing tag includes the same text as the opening tag, but with an added forward-slash. The first tag is the starting tag and the second is the final tag. There are also tags that are self-closing, i.e., they do not need a closing tag, e.g., and
.

The tags needed to define the format of an HTML document are:

<|human|>.

Important tags used to define the format of an HTML document are:

<|human|>.

Among others, these tags are required:

<|human|><|human|>.

This A-to-Z list of HTML tags is a useful reference tool, either for a novice just getting into the world of web development or a veteran developer wanting a quick reference guide. HTML is simple, and a versatile language. With the ability to master these tags, it can develop interesting, accessible, and search-engine friendly websites that are attractive as well as perfectly functional (GeeksforGeeks, 2019).

Bookmark Management

Purpose

Bookmarks give the user the ability to store their favorite or frequently visited pages so that they can access them easily. Here we apply techniques to add, delete and manipulate bookmarks.

The internet has become an indispensable information gathering, researching, and entertainment tool in the digital era. There is so much to read online that it becomes so easy to lose track of the web sites, articles, and other resources they find most valuable. Here the bookmarking provides a simple but strong feature that has been the keynote of web browsers decades ago. Bookmarking refers to the act of visiting a webpage or web site and storing it as a reference. By bookmarking a page, they are adding a shortcut to the particular URL, so that they can easily retrieve it again in the future without needing to re-search. The bookmarks may be saved on a local device or on more than two devices through cloud-based applications. Bookmarking saves their time as they do not need to search the same sites or pages yet again. Bookmarking will allow them to classify their stored information into a folder, tags, or categories to be able to locate what they need when they need it (Elmore, 2025).

The digital age has seen their method of saving and accessing web sites change which used to be the classic bookmarks in our favorite books, now it is the virtual bookmarks in our internet browsers. Nonetheless, bookmarks and favorites are interchangeable terms that confuse people who use the internet. The basic concept of bookmarks and favorites will help the reader to have a clear picture of what a bookmark and favorites are and the role they play in web browsers. Both bookmarks and favorites are the functions of the web browsers that enable customers to save certain webpages that they want to revisit later. They basically serve as shortcuts to certain online information so that customers can easily get to their favorite web sites without necessarily remembering or searching after the URLs each time. In browsers such as Google Chrome, Mozilla Firefox, and Safari, the equivalent would be bookmarks, and for Internet Explorer or Microsoft Edge, it would be favorites. However, the concept is the same even though it is called different things (Ellison, 2024).

Code Implementation

We have a Bookmark class and a Bookmark Manager class to manage the storage and retrieval of bookmarks:

C Sharp Implementation

```

1  using System;
2  using System.Collections.Generic;
3  using System.Diagnostics;
4  using System.Drawing;
5  using System.IO;
6  using System.Text.Json;
7  using System.Text.Json.Serialization;
8  using System.Windows.Forms;
9
10 #region Models + Persistence
11
12 internal class Bookmark
13 {
14     public string Name { get; set; } = "";
15     public string Url { get; set; } = "";
16     [JsonIgnore] public override string ToString() => string.IsNullOrEmpty(Name) ? Url : $"{Name} - {Url}";
17 }
18
19 internal static class BookmarkStore
20 {
21     private static readonly string Dir = Path.Combine(
22         Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData),
23         "MiniBookmarksGui");
24     private static readonly string FilePath = Path.Combine(Dir, "bookmarks.json");
25
26     public static string StoragePath => FilePath;
27
28     public static List<Bookmark> Load()
29     {
30         try
31         {
32             Directory.CreateDirectory(Dir);
33             if (!File.Exists(FilePath))
34             {
35                 // Seed with an empty list on first run
36                 Save(new List<Bookmark>());
37                 return new List<Bookmark>();
38             }
39         }
40     }
41 }

```

Explanation

Bookmark Class

Saves details about a bookmark include its name and the URL.

The creation of bookmarks is easy, however, in the long run maintaining a large number of bookmarks will be a problem. The name proposed by the browser as the bookmark is the same as the webpage name, which can be descriptive or not of the page. Name the bookmark descriptively when adding the bookmark, so that, in the future, at the first look, they will know where the bookmark leads. When they store all the bookmarks in one folder, they will have a disorganized library. Then, they need to have different folders where they can store various bookmarks. Firefox allows them to use the bookmark organizing tool (see the image below) by choosing Organize Bookmarks in the Bookmarks menu. Most users create back-ups of their important files and forget to create back-ups of their bookmarks. The pages are recorded in the

browser history. The browser history enables them to visit previously visited pages even when the pages have not been saved as bookmarks (Anon, n.d.).

Bookmark Manager Class

Processes reading and writing bookmarks to a JSON file. It allows us to add, delete, and save bookmarks in a file in the user App Data folder.

It is easy to synchronize bookmarks offline. Create, organize and otherwise manage bookmarks in the chrome. Bookmarks API. Import and export bookmarks between browsers through a Json file. Click on chrome history. Json or chrome bookmarks. Json. Import a local bookmarks directory directly out of a Json source. The output is the Json is a package which converts browser bookmark files, either as a module or as a cli. Export their bookmarks, or a subset of their bookmarks, by selecting a parent folder. convert an exported chrome bookmarks HTML file using JavaScript. Gives them the ability to drag bookmarks to a particular folder. So it has read through and experimented with the code in merging the bookmark Json files of chrome and edge chromium into (Guide of Greece, n.d.).

History Management

Purpose

The browser must follow the navigation history of user, and it supports features such as Back and Forward. This section describes how we handle history so that multiple entries with the same duplicate are not added.

One tab, on which they are on the home page, they can visit another URL, get back in the history number of steps, or forward in the history number of steps. The challenge is to create a data format and provide the application for going to a URL, beginning with the home page, and going forwards and backwards in history. Store the present URL and the past URLs in the back button. Open the browser with a specified homepage. Press the home page into the back Stack to make it the first page. Clear the forward Stack since forward history is invalidated by visiting a new page. Pop each component of forward Stack. Move the new URL to the back Stack and it becomes the current page (GeeksforGeeks, 2022).

backStack



forwardStack



1. **visit("google.com")**
2. **visit("facebook.com")**
3. **visit("youtube.com")**



User experience has become the most important thing in a modern web application and navigation control is a vital part of experience. A frequent fact that developers come across is how to deal with the back button of the browser used in React apps. Unlike traditional websites, react applications frequently rely on client-side routing, which can create some unforeseen results as users back-traverse their history. This may lead to an entangled experience otherwise handled appropriately. Back-on-browse followed by forward-on-browse, or vice versa, does not execute their code and does not perform a use effect or cleanup. As the user browses the session history, this pop state event of the Window interface is activated as the currently active history entry is modified. It replaces the existing history entry with the previous page the user was on or,

when history.PushState has been called to insert a history entry onto the history stack, the history entry is used instead (Aditya Singh Dabra, 2024).

Code Implementation

We follow up the History lesson to chart and sail through history:

C Sharp Implementation

```
1  using System;
2  using System.Collections.Generic;
3  using System.Diagnostics;
4  using System.Drawing;
5  using System.IO;
6  using System.Text.Json;
7  using System.Text.Json.Serialization;
8  using System.Windows.Forms;
9
10 #region Persistence Models
11
12 internal class HistoryEntry
13 {
14     public string Url { get; set; } = "";
15     public DateTime VisitedAt { get; set; } = DateTime.UtcNow;
16
17     [JsonIgnore]
18     public override string ToString() => Url;
19 }
20
21 internal class HistorySnapshot
22 {
23     public List<HistoryEntry> Entries { get; set; } = new();
24     public int CurrentIndex { get; set; } = -1;
25 }
26
27 internal static class HistoryStore
28 {
29     private static readonly string Dir = Path.Combine(
30         Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData),
31         "MiniHistoryGui");
32     private static readonly string FilePath = Path.Combine(Dir, "history.json");
33
34     public static string StoragePath => FilePath;
35
36     public static HistorySnapshot Load()
37     {
38         try
39         {
```

Explanation

History Class

Add To History method inserts a URL in the history list, no consecutive ones. Go Back and Go Forward functions can enable one to navigate the history and the functions Can Go Back and Can Go Forward can be used to check whether one can take any step (forwards or backward).

They will access the history stack of JavaScript through the History object. All web browsers will have a stack of the history of the webpages or websites visited during the browsing session. They must use the history object in JavaScript in order to access this history stack. History object holds the history of the browser. The history object stores the URLs of the visited pages as a stack. The historical object can be managed / accessed in various ways. This is how the next URL in the history list is loaded. This does exactly what the next button in the browser does. No parameters and nothing will be returned. This technique is to load the last page in history list. This has the same functionality as the back button on the browser. No parameters and nothing will be returned (GeeksforGeeks, 2022).

Win Forms UI Setup

Purpose

The user interface (UI) has access to the browser, its buttons, input fields, and display panels. The UI needs to be responsive.

The human-computer interface (HRI) of a device is the interface with the user. This may consist of display interfaces, keyboards, a mouse and the features of a desktop. It is also the way a user engages with an application or a site, through visual and audio, like type fonts, icons, buttons, animation and sounds. A good UI is one that follows design principles that allow users to navigate through it and use it in their desired ways with ease. Web and mobile apps are relied upon in many real-life businesses. This has seen companies put more emphasis on UI in order to enhance the overall user experience. UI and web designing is no longer only about hard skills such as coding. They also need information on patterns of user interaction design, and access to keep interfaces easy to use by all and not too complex (Churchville, 2021).

Code Implementation

The following is the code of establishing the UI within the main form:

C Sharp Implementation

```
1 using System;
2 using System.Collections.Generic;
3 using System.Diagnostics;
4 using System.Drawing;
5 using System.IO;
6 using System.Linq;           // LINQ
7 using System.Net;
8 using System.Net.Http;
9 using System.Threading.Tasks;
10 using System.Windows.Forms;
11 using Microsoft.Data.Sqlite;
12
13 #region Models
14
15 record User(int Id, string Name);
16 record Bookmark(int Id, int UserId, string Name, string Url);
17 record HistoryEntry(int Id, int UserId, string Url, DateTime VisitedAt);
18 record SettingsRow(int UserId, string Homepage);
19
20 #endregion
21
22 #region Data Access (SQLite)
23
24 static class Db
25 {
26     private static readonly string Dir =
27         Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData), "MiniBrowserDb");
28     private static readonly string PathDb = System.IO.Path.Combine(Dir, "browser.db");
29     public static string ConnectionString => $"Data Source={PathDb};Cache=Shared";
30
31     public static void Ensure()
32     {
33         Directory.CreateDirectory(Dir);
34         var firstCreate = !File.Exists(PathDb);
35         using var con = new SqliteConnection(ConnectionString);
36         con.Open();
37         using var cmd = con.CreateCommand();
38         cmd.CommandText = @"
```

Explanation

The UI includes:

- A typing URL address bar.
- An A Go button triggered the HTTP request.
- A page title to show the title of the page.
- The content to be displayed in a raw HTML format.
- A links sidebar which updates with the top five links on the page.

Main Win Forms Browser Interface (Idle State)

```
1  using System;
2  using System.Windows.Forms;
3
4  namespace MiniWinFormsBrowser
5  {
6      internal static class Program
7      {
8          [STAThread]
9          static void Main()
10         {
11             ApplicationConfiguration.Initialize();
12             Application.Run(new MainForm());
13         }
14     }
15 }
```

Top 5 Links Extracted after HTTP Request

<https://en.wikipedia.org/>

<https://commons.wikimedia.org/>

<https://www.wiktionary.org/>

<https://www.wikibooks.org/>

<https://www.wikivoyage.org/>

```
1 using System;
2 using System.Collections.Generic;
3 using System.Diagnostics;
4 using System.Linq;
5 using System.Net.Http;
6 using System.Text;
7 using System.Threading;
8 using System.Threading.Tasks;
9 using System.Web;
10 using System.Windows.Forms;
11 using HtmlAgilityPack;
12
13 namespace MiniWinFormsBrowser
14 {
15     public class MainForm : Form
16     {
17         // --- Controls ---
18         private readonly TextBox txtAddress = new TextBox();
19         private readonly Button btnGo = new Button();
20         private readonly Button btnBack = new Button();
21         private readonly Button btnForward = new Button();
22         private readonly Button btnReload = new Button();
23
24         private readonly Label lblTitle = new Label();
25         private readonly ListBox lstTopLinks = new ListBox();
26         private readonly TextBox txtRawHtml = new TextBox();
27
28         private readonly StatusStrip statusStrip = new StatusStrip();
29         private readonly ToolStripStatusLabel statusLabel = new ToolStripStatusLabel();
30         private readonly ToolStripStatusLabel httpStatusLabel = new ToolStripStatusLabel();
31
32         // --- Networking & State ---
33         private readonly HttpClient http = new HttpClient(new HttpClientHandler
34         {
35             AllowAutoRedirect = true,
36             AutomaticDecompression = System.Net.DecompressionMethods.All
37         });
38     }
39 }
```

Conclusion

This project is a full-fledged introduction to working with HTTP requests, HTML parsing, and creating a basic web browser application. We could implement the features in stages, as we could have broken down the requirements into small portions, including HTTP requests, HTML

parsing, bookmarks, and history management. WinForms UI is very user friendly and the logic behind it can make sure that the browser itself is usable, such as navigating, bookmarking, and tracking history. The project has contributed to the knowledge of C# programming and web development concepts, and can be extended with capabilities such as multi-user support, improved parsing algorithms and improved error handling.

Flask is a lightweight pythonic web application framework. It is commonly referred to as a micro-framework in that it offers just enough functionality to build the web development without extraneous complexity. Flask does not have in-built functionalities such as authentication, an administration panel or anything that Django has, but rather it keeps it to a minimum and allows them to add functionality when they need it. This Flask tutorial includes setup and installation as well as more advanced topics such as authentication and integration and deployment of a database. They will also get ideas of Flask projects, frequently asked questions and interview questions to enable them prepare job interviews. They will also talk about how to install Flask, to get them ready to begin building web applications. Flask templates are HTML files that can be used to produce dynamically generated web pages. Jinja2, a highly capable templating engine, is used by Flask to incorporate Python logic within templates. CSS, JavaScript and images are stored separately to improve the user interface (GeeksforGeeks, 2023).

References

- Cloudless.com. (2019). Developing Desktop Applications with Windows Forms in .NET. [online] Available at: <https://clouddevs.com/dot-net/desktop-applications-with-windows-forms/>.
- GeeksforGeeks (2020). Fetch API in JavaScript. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/javascript/javascript-fetch-method/>.
- Improvidence (2025). HTTP Client in C#: Best Practices for Experts. [online] Medium. Available at: <https://medium.com/@iamprovidence/http-client-in-c-best-practices-for-experts-840b36d8f8c4>.
- GeeksforGeeks (2016). Implementing Web Scraping in Python with BeautifulSoup. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/python/implementing-web-scraping-python-beautiful-soup/>.
- Data Journal (2024). How to Parse HTML With Python. [online] Medium. Available at: <https://medium.com/@datajournal/how-to-parse-html-with-python-94495c11bc96>.
- GeeksforGeeks (2019). HTML Tags A to Z List. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/html/html-tags-a-to-z-list/>.

- Elmore, J. (2025). The Power of Bookmarking: Unlocking Efficient Browsing and Organization. [online] The Techy Life. Available at: <https://thetechylife.com/what-is-the-importance-of-bookmark/>.
- Ellison, J. (2024). What's the Difference Between Bookmarks and Favorites: A Comparison – Blinks and Buttons. [online] Blinks and Buttons. Available at: <https://blinksandbuttons.net/whats-the-difference-between-bookmarks-and-favorites/>.
- Anon, (n.d.). Bookmarks and browsing history | Student's digital skills. [online] Available at: <https://blogs.helsinki.fi/students-digital-skills/1-introduction-to-the-use-of-computers/1-3-the-internet-and-www/bookmarks-and-browsing-history/>.
- Guide of Greece. (n.d.). Guide Of Greece. [online] Available at: <https://guideofgreece.com/>.
- GeeksforGeeks (2022). Design Browser History. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/design-custom-browser-history-based-on-given-operations/>.
- Aditya Singh Dabra (2024). How to handle browsers back/forward Navigation. [online] Medium. Available at: <https://medium.com/@adityasinghdabhra/how-to-handle-browsers-back-forward-navigation-b2adc56ba42a>.
- GeeksforGeeks (2022). How to access history in JavaScript? [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/javascript/how-to-access-history-in-javascript/>.
- Churchville, F. (2021). What is user interface (UI)? Definition from Search App Architecture. [online] Search App Architecture. Available at: <https://www.techtarget.com/searchapparchitecture/definition/user-interface-UI>.
- GeeksforGeeks (2023). Flask Tutorial. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/python/flask-tutorial/>.